

Algoritmo Genético

quarta-feira, 2 de julho de 2025 22:07

Inteligência gerada por algoritmos genéticos:

👉 É quando você usa **princípios de evolução (seleção, cruzamento, mutação)** pra “evoluir” soluções e encontrar a melhor.

📍 Como funciona?

- Cria várias soluções aleatórias (população).
- Avalia quais são melhores (fitness).
- Cruza e muta as melhores pra gerar novas soluções.
- Repete até achar uma bem otimizada.

✅ Em resumo:

É IA que aprende “por tentativa e erro evolutiva”, copiando a natureza pra achar soluções cada vez melhores.

Problema do Caixeiro Viajante:

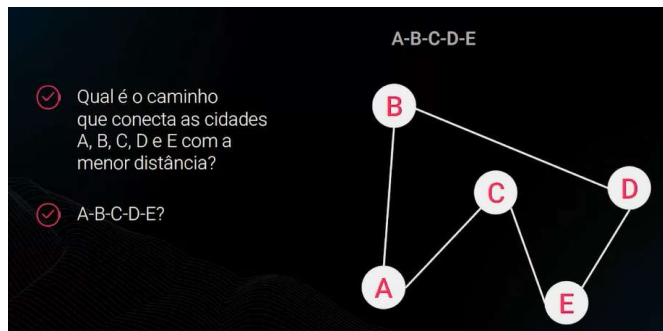
É encontrar o menor caminho que passa por todas as cidades (A, B, C, D, E) uma vez cada e retorna ao início.

Como o algoritmo genético resolve:

- OCria várias rotas aleatórias entre as cidades.
- Avalia quais rotas são mais curtas.
- Combina as melhores rotas e faz pequenas mudanças (mutações).
- Repete o processo até encontrar uma rota bem curta.

Resumo:

O algoritmo genético busca o caminho mais curto testando e melhorando rotas a cada geração, sem precisar tentar todas as possibilidades.



Soluções Exatas

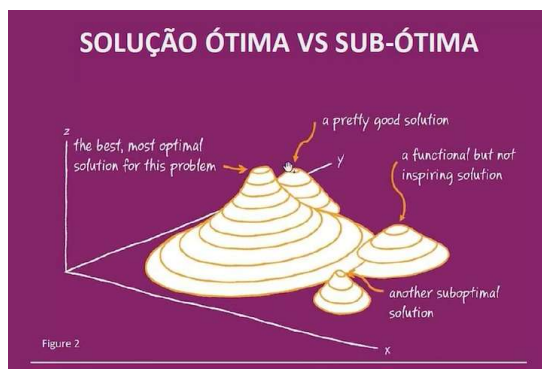
- **O que são:** Encontram a resposta perfeita.
- **Vantagem:** Garantem o melhor resultado.
- **Desvantagem:** Podem ser lentas em problemas grandes.

Soluções Heurísticas

- **O que são:** Encontram uma resposta boa, mas não garantem ser a melhor.
- **Vantagem:** São rápidas e funcionam pra problemas grandes.
- **Desvantagem:** A solução pode não ser perfeita.

Soluções Ótima x Sub-Ótima

- **Solução ótima:** É o topo mais alto do gráfico. Representa o melhor resultado possível para o problema – ou seja, o melhor dos melhores, o objetivo final.
- **Soluções sub-ótimas:** São “picos” menores. Elas são boas, funcionais, mas não são as melhores possíveis. Servem, mas podem ser melhoradas.
- **Por que isso importa?** Em muitos problemas complexos, é difícil ou demorado achar a solução ótima. Por isso, muitas vezes aceitamos soluções sub-ótimas (boas o suficiente), especialmente usando heurísticas.



Claro! Aqui vai a explicação com emoticons, daquele jeito bem informal:

Força Bruta vs. Método Analítico

Força Bruta 🐢

- Testa **todas as possibilidades** até achar a resposta.
- Funciona sempre, mas pode ser **muito lento** em problemas grandes. 🐢
- Exemplo: tentar todas as combinações de senha pra descobrir a certa. 🗝️

Método Analítico 🧠

- Usa **fórmulas, lógica ou matemática** pra chegar direto na solução, sem testar tudo.
- Normalmente, é **bem mais rápido**. ⚡
- Exemplo: resolver uma equação pra encontrar a resposta. 📐

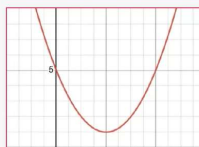
Resumo:

- **Força bruta:** tenta tudo 🐢, garante resposta ✅, mas pode demorar ⏰.
- **Analítico:** vai na lógica 🧠, resolve mais rápido ⚡ (quando possível).

MÉTODO ANALÍTICO

Como resolvemos?

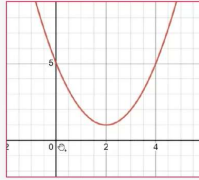
- Existe um método exato e analítico através da derivada da função



MÉTODO ANALÍTICO

Como resolvemos?

- ✓ Existe um método exato e analítico através da derivada da função
- ✓ Nem todos os problemas podem ser resolvidos de maneira analítica por serem muito complexos, não-lineares, ou não-diferenciáveis



FORÇA BRUTA

Como resolvemos?

- ✓ Por força bruta, podemos testar literalmente todas as combinações de cidades e encontrar a combinação que tem a menor distância

ABCEFD, Distance: 10
ABDCE, Distance: 19
ABDEC, Distance: 19
ABECD, Distance: 24
ABFDC, Distance: 21
ACBDE, Distance: 18
ACDBE, Distance: 23
ACDEB, Distance: 21
ACEDB, Distance: 26
ACEBD, Distance: 19
ADBCE, Distance: 21
ADBEC, Distance: 26
ADCEB, Distance: 23
ADECB, Distance: 24
ADEBC, Distance: 23
ADECB, Distance: 19
AEBDC, Distance: 23
AEBDC, Distance: 23
AECDB, Distance: 21
AECDB, Distance: 19
AECDB, Distance: 18
AECDB, Distance: 16
Total of paths: 24
Optimal Route (A, B, C, D, E)
Minimum Distance: 16

O que é Teoria da Evolução?

A **teoria da evolução** diz que os seres vivos mudam ao longo do tempo, se adaptando ao ambiente. Quem se adapta melhor, sobrevive e passa suas características pra frente.

- **Quem criou?** Charles Darwin ficou famoso por isso.
- **Como funciona?**
 - Surgem mudanças (mutações) nos seres vivos.
 - Os mais adaptados têm mais chance de sobreviver e ter filhos.
 - Assim, essas características “boas” vão se espalhando na população.

Algoritmos Bio-inspirados?

São algoritmos que se inspiram na **natureza** ou nos **seres vivos** pra resolver problemas.

Eles imitam processos naturais, tipo evolução, comportamento de formigas, enxames de abelhas, etc.

- **Exemplos:**
 - **Algoritmos Genéticos:** imitam a evolução das espécies.
 - **Algoritmo das Formigas:** copia como as formigas acham o melhor caminho pra comida.
 - **PSO (Enxame de Partículas):** imita o movimento de bandos de pássaros ou peixes.

Resumo:

- Usam ideias da biologia pra criar soluções criativas e eficientes pra problemas difíceis.

O que é Teoria da Evolução por Seleção Natural?

É a ideia de que os seres vivos **evoluem** porque alguns têm características que ajudam mais na sobrevivência e reprodução. Quem tem essas vantagens, vive mais e deixa mais descendentes. Aos poucos, essas características “boas” vão ficando mais comuns.

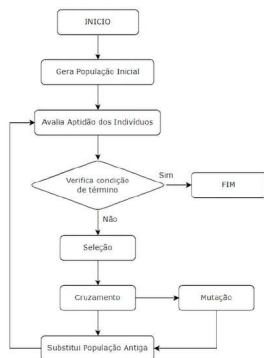
- **Como funciona?**
 - **Variedade:** Nem todo mundo nasce igual. Sempre tem diferenças entre os indivíduos.
 - **Competição:** O ambiente tem recursos limitados, então rola uma disputa pra sobreviver.
 - **Seleção natural:** Quem é mais “adaptado” ao ambiente sobrevive e tem mais filhos. Quem não se adapta, dança.
 - **Herança:** Os filhos herdam as características dos pais.

Resumo informal:

Os mais “fortes”, “espertos” ou “adaptados” passam seus genes pra frente, e a espécie vai mudando com o tempo.

Como evoluir características em um algoritmo?

1. Um indivíduo é uma possível solução para o problema
2. Criamos novas soluções a partir de soluções existentes na população atual
3. Mutação introduz variações aleatórias nas soluções
4. Selecionamos as melhores soluções usando uma função de aptidão
5. Selecionamos as melhores soluções por meio do valor de aptidão
6. Novas soluções podem resolver melhor o problema do que geração anterior



Tipos de Codificação e Quando Usar

1. Codificação Binária

- **Como é:** Representa soluções como uma sequência de 0s e 1s (bits).
- **Quando usar?** Problemas simples, tipo ligado/desligado, sim/não, escolha discreta (ex: problema da mochila).
- **Por quê?** Fácil de implementar, perfeito pra variáveis binárias.

2. Codificação Real

- **Como é:** Usa números reais (decimais) direto na solução.
- **Quando usar?** Problemas com valores contínuos, ajustes de parâmetros, otimização de funções matemáticas.
- **Por quê?** Mais preciso pra lidar com números de verdade.

3. Codificação Combinatória

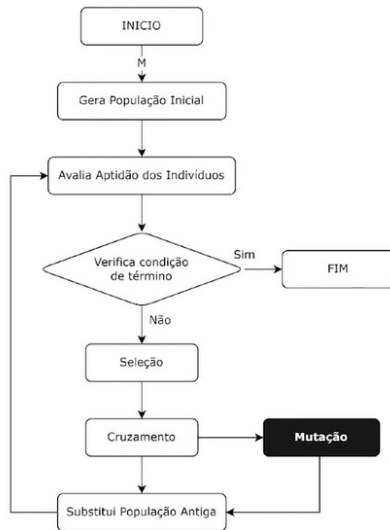
- **Como é:** Solução é uma sequência/ordem de elementos.
- **Quando usar?** Quando a ordem importa, tipo caixeiro viajante, escalonamento de tarefas.
- **Por quê?** Precisa saber a ordem em que as coisas acontecem.

4. Codificação Híbrida

- **Como é:** Mistura dois ou mais tipos (binário, real, combinatório).
- **Quando usar?** Problemas complexos que têm diferentes tipos de variáveis juntos.
- **Por quê?** Permite atacar partes diferentes do problema do jeito mais eficiente.

Resumo prático:

- **Binária:** Sim/não, ligado/desligado
- **Real:** Números contínuos
- **Combinatória:** Ordem/sequência
- **Híbrida:** Mistura de tudo acima



1. **Início** ●
Começa o algoritmo.
2. **Gera População Inicial** 🎲
Cria várias soluções aleatórias (ou usando hotstart, se tiver alguma dica).
3. **Avalia Aptidão dos Indivíduos** 📏
Mede o “quão boa” é cada solução (nota de fitness pra cada um).
4. **Verifica Condição de Término** 🛑
Checa se já deu pra parar (atingiu o objetivo ou chegou no limite de gerações).
 - a. **Se sim:** Fim do algoritmo 🏁
 - b. **Se não:** Continua o processo
5. **Seleção** 🎯
Escolhe os melhores da galera pra serem “pais” da próxima geração.
6. **Cruzamento (Crossover)** 🔄
Mistura genes dos pais pra criar filhos novos.
7. **Mutação** ✨
Dá aquela mexida aleatória nos filhos pra garantir variedade.
8. **Substitui População Antiga** 🔄
Troca a geração velha pelos novos indivíduos e volta pro início do ciclo.